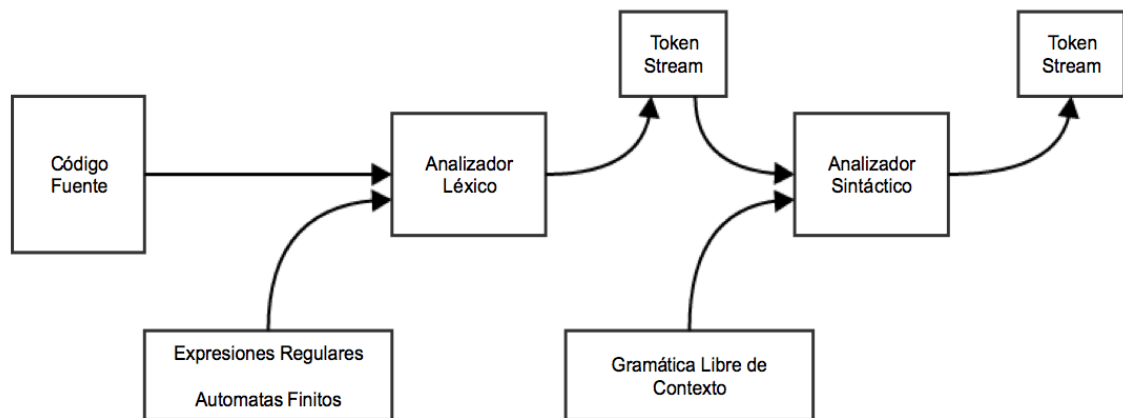


TP N°1 AUTOMATAS - GRUPO 3



Comisión: K2002

Grupo: 3

Profesora: Roxana Leituz

Integrantes: Uriel Gregorio Cwirenbaum, Bruno Chiurazzi, Francisco Squeri, Martin Ignacio Sosa, Mariana Markiw

Materia: Sintaxis y semántica de los lenguajes

Año: 2026

Ejercicio 1:

Dada una cadena que contenga varios números que pueden ser decimales, octales o hexadecimales, con o sin signo para el caso de los decimales, separados por el carácter '@', reconocer los tres grupos de constantes enteras, indicando si hubo un error léxico, en caso de ser correcto contar la cantidad de cada grupo.

Para la resolución se optó por implementar un autómata finito determinístico (AFD), mediante una matriz de transición (TT).

El código se diseñó en dos etapas principales. Primero, la función `esPalabra()` implementa el autómata usando una matriz de 8 filas (los estados) y 7 columnas (el alfabeto). Esta función recorre la cadena ingresada de principio a fin; si detecta algún error, como un símbolo que no corresponde, cae en el estado de fallo y rechaza toda la cadena.

Solo si esta revisión es exitosa y el autómata termina en un estado de aceptación, el programa avanza a la segunda etapa. Donde, la función `clasificarYContar()` usa `strtok` para ir separando la cadena cada vez que encuentra un arroba '@'. Como el autómata ya se encargó de verificar que todos los números están bien escritos, el programa simplemente los clasifica mirando cómo empiezan: si empieza con 0x es hexadecimal, si empieza solo con 0 es octal, y el resto se cuenta como decimal. Una vez procesados todos, se imprimen los totales por pantalla.

Alfabeto del autómata:

$$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, x, A, B, C, D, E, F, @\}$$

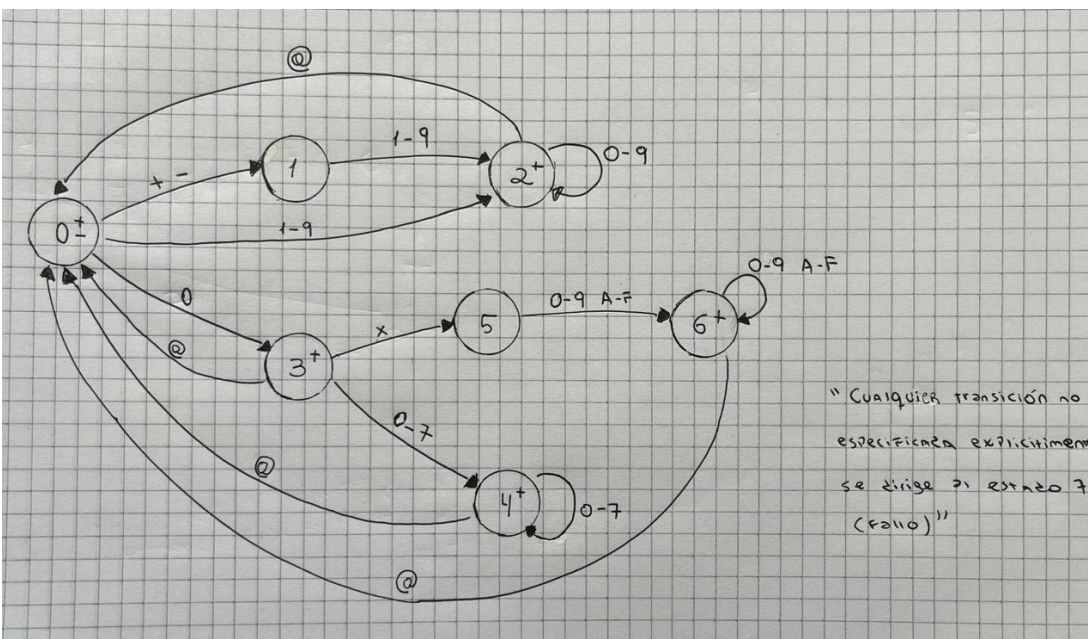
Tabla de transición:

TT	+ -	0	1-7	8-9	A-F	x	@
0 + -	1	3	2	2	7	7	0
1	7	7	2	2	7	7	7
2+	7	2	2	2	7	7	0
3+	7	4	4	7	7	5	0
4+	7	4	4	7	7	7	0
5	7	6	6	6	6	7	7
6+	7	6	6	6	6	7	0
7	7	7	7	7	7	7	0

Estados:

0	Inicial
1	Signo
2	Decimal +
3	Cero Inicial +
4	Octal +
5	Prefijo Hexa
6	Hexadecimal +
7	Fallo / Error

Autómata:



Ejemplo de uso (EJ1):

Funcionamiento con el ejemplo propuesto en el código (+45@077@0x1F@120)

1. Lee el inicio de la cadena
2. Lee el +, espera dígitos
3. Lee 4, valida decimal
4. Lee 5, valida decimal
5. Lee @, separador, vuelve al principio
6. Lee 0, todavía no decide
7. Lee 7 sin signo, eso implica que es un octal
8. Lee 7, también octal
9. Lee @, separador nuevamente
10. Lee 0, todavía no decide
11. Lee x, detecta dígitos hexa y los espera
12. Lee 1, valida hexadecimal
13. Lee F, valida hexadecimal
14. Lee @, separador nuevamente
15. Lee 1, valida decimal
16. Lee 2, valida decimal
17. Lee 0, valida decimal
18. Fin de cadena

```
Ingrese la cadena de numeros separados por '@' (ej: +45@077@0x1F@120): +45@077@0x1F@120
Resultado: La cadena SE RECONOCE (pertenece al lenguaje).

--- Conteo de constantes enteras ---
Decimales:      2
Octales:        1
Hexadecimales: 1
```

Donde:

--- Conteo de constantes enteras ---

Decimales: 2 → (+45, 120)

Octales: 1 → (077)

Hexadecimales: 1 → (0x1F)

Ejemplo error por fallo de reconocimiento del autómata:

```
Ingrese la cadena de numeros separados por '@' (ej: +45@077@0x1F@120): +45@077@0x1F@12f
Resultado: ERROR LEXICO (la cadena NO se reconoce).
```

No se declaró correctamente el último número si hubiera sido hexadecimal

Ejercicio 2:

Debe realizar una función que reciba un carácter numérico y retorne un número entero.

Para la resolución de este ejercicio usamos una función (atoi) que estaba en el libro Kernighan - Ritchie (Sección 2.7, Página 47) y le agregamos algo de código para la eliminación de espacios en el carácter numérico.

La función atoi convierte cada carácter numérico en su versión entera, y lo multiplica por 10 dependiendo de la posición en la que esté ese carácter.

Después, en el main solicitamos el ingreso del número entero y lo mostramos en pantalla.

Ejemplo de uso (EJ2):

Alfabeto del autómata:

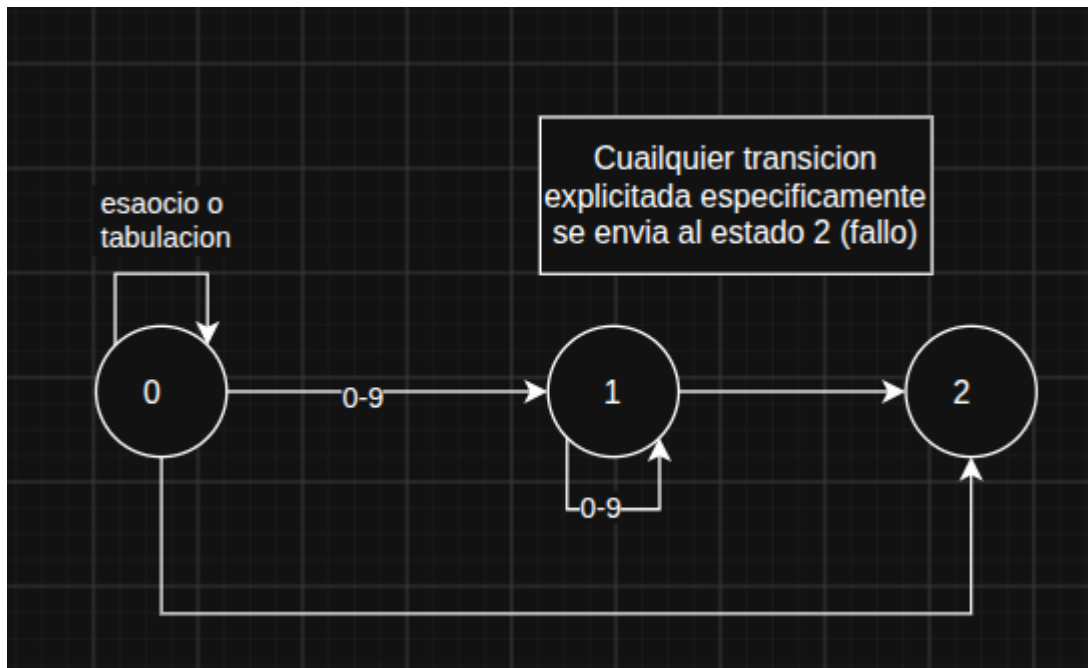
$$\Sigma = \{0,1,2,3,4,5,6,7,8,9,+,-,' '(espacio),/t (tabulación)\}$$

Estados: (estados de aceptación: 2 y 3)

0	Inicial
1	Número +
2	Fallo

Tabla de transición:

TT	esp	tab	0-9	otro
0	0	0	1	2
1+	2	2	1	2
2	2	2	2	2



ejemplo con cadena “1234”:

Funcionamiento con el ejemplo 1234:

1. El programa solicita el ingreso de un número entero.
2. Se ingresa la cadena 1234.
3. La función esVálido comienza a recorrer la cadena desde el primer carácter.
4. Lee 1 y verifica que sea un dígito.
5. Lee 2 y verifica que sea un dígito.
6. Lee 3 y verifica que sea un dígito.
7. Lee 4 y verifica que sea un dígito.
8. Llega al final de la cadena sin encontrar caracteres inválidos.
9. La función esVálido retorna verdadero.
10. El programa llama a la función atoi().
11. atoi() inicializa el valor de n en 0.
12. Lee el carácter 1 y calcula $n = 10 * 0 + 1$, por lo que $n = 1$.
13. Lee el carácter 2 y calcula $n = 10 * 1 + 2$, por lo que $n = 12$.
14. Lee el carácter 3 y calcula $n = 10 * 12 + 3$, por lo que $n = 123$.
15. Lee el carácter 4 y calcula $n = 10 * 123 + 4$, por lo que $n = 1234$.
16. Llega al final de los caracteres numéricos.
17. La función atoi() retorna 1234.
18. El programa muestra en pantalla:

```

Ingrese un numero entero: 1234
El numero ingresado es: 1234
  
```

ejemplo por fallo de reconocimiento del autómata:

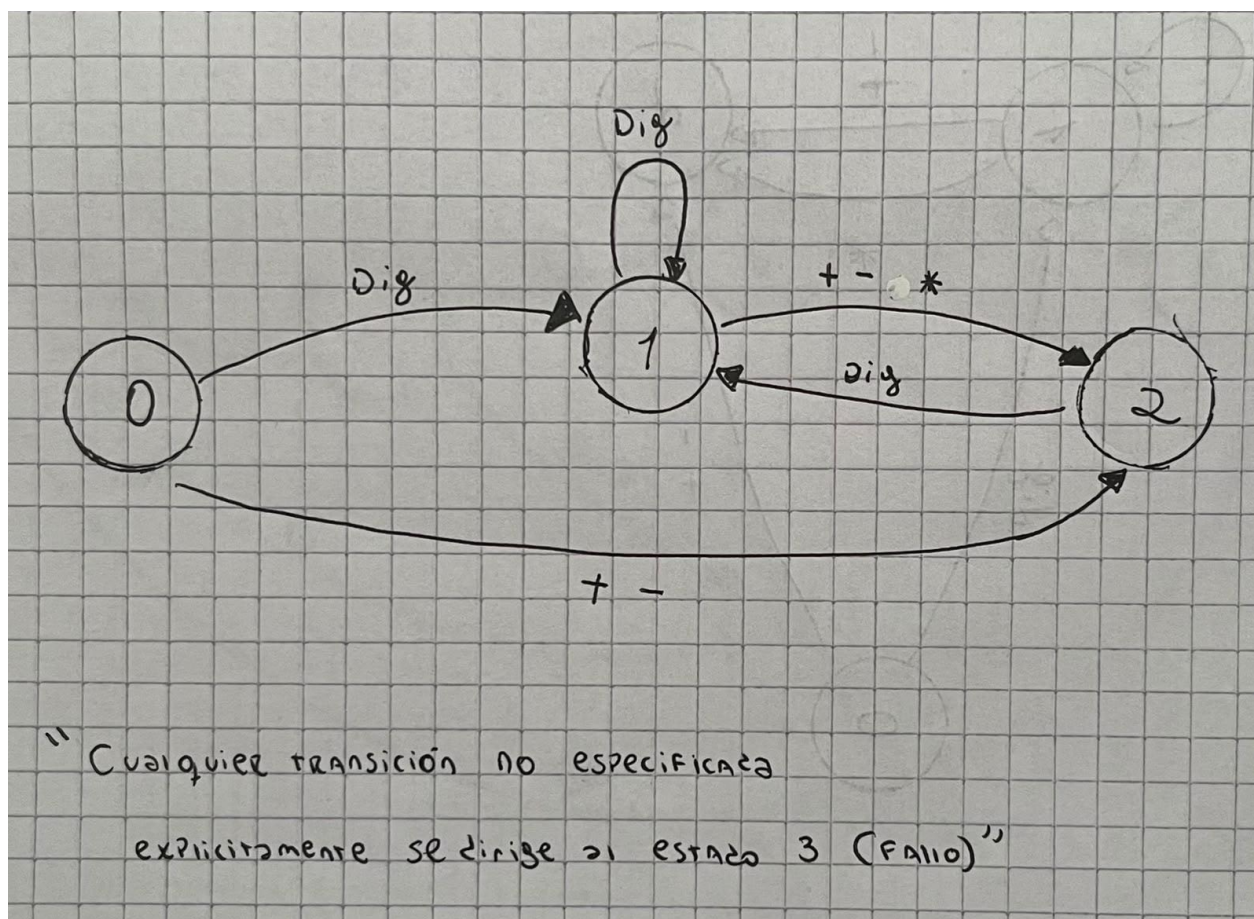
```
Ingrese un numero entero: */k123a
Error: la cadena no fue reconocida por el automata.
```

Ejercicio 3:

Ingresa una cadena que represente una operación simple con enteros decimales y obtener su resultado, se debe operar con $+$, $-$ y $*$. Ejemplo $= 3+4*7+3-5 = 29$. Considerando que el alfabeto son los números decimales, signos $+$, $-$ y $*$ valide con un autómata previo a la operación que dicha cadena que representa la operación pertenezca al lenguaje.

Para la resolución de este ejercicio usamos un autómata para la validación de la cadena, tal como lo pide el programa y después separamos el código en dos partes: una parte se ocupa de la transición de la cadena de infijo a postfijo (polaco inverso) y la otra parte se ocupa de calcular el número final, usando una versión de calculadora con pilas para calcular números en polaco inverso

Autómata: (Dig = Dígito)



La primera parte del código para pasar de infijo a postfijo opera así:

/*La función de transformación de infijo a postfijo básicamente recorre la cadena, apila los números en una pila y los operadores en otra,

cuando encuentra un operador con precedencia menor o igual al operador en la cima de la pila, desapila los operadores y los agrega a la salida

hasta que encuentra un operador con menor precedencia o la pila queda vacía. Al final, vacía la pila de operadores a la salida.*/ (comentario sacado del código en el trabajo)

Después de esta transformación, la segunda parte del código toma esta cadena en postfijo y, usando pilas, va apilando los números hasta que se topa con un operador. En el caso de toparse con un operador el programa desapilará los últimos dos dígitos y operará con estos dependiendo del operador.

Esto se repetirá hasta que quede un solo dato en la pila, el cual será el resultado de la operación.

Ejemplo de uso (3+4*7+3-5):

Parte conversión a notación polaca:

1. Recibe la operación
2. Lee 3, lo copia
3. Lee +, lo apila
4. Lee 4, lo copia
5. Lee *, compara precedencia y lo apila
6. Lee 7, lo copia,
7. Lee +, compara precedencia, desapila *, desapila +, apila +
8. Lee 3, lo copia
9. Lee -, compara precedencia, desapila +, apila -
10. Lee 5, lo copia
11. Fin de la cadena, desapila -

Parte solución de operación:

1. Recibe token 3, lo apila
2. Recibe token 3, lo apila
3. Recibe token 7, lo apila
4. Recibe token *, desapila 4 y 7, realiza la operación 4*7 y apila el resultado
5. Recibe token +, desapila 28 y 3, realiza la operación 3+28 y apila el resultado
6. Recibe token 3, lo apila
7. Recibe token +, desapila 31 y 3, realiza la operación 31 + 3 y apila el resultado
8. Recibe token 5, lo apila
9. Recibe token -, desapila 34 y 5, realiza la operación 34-5 y apila el resultado
10. Realiza un return pop(), que devuelve 29

```
Ingrese la operacion (ej: 3+4*7+3-5): 3+4*7+3-5
Notacion Polaca Inversa: 3 4 7 * + 3 + 5 -
Resultado: 29
```

Ejemplo error por no reconocer la cadena:

```
Ingrese la operacion (ej: 3+4*7+3-5): 2+3-5*3+(1+1)
Error: La cadena no pertenece al lenguaje.
```